

生産管理システム データベース改善実装計画

📄 概要

本ドキュメントは、提案されたER図（T01-T09）と現在のシステムとの差分分析に基づく、段階的なデータベース改善実装計画です。

作成日: 2025-12-11 対象システム: 生産管理システム (pm)

🔗 実装優先度と概要

💧 優先度：高（コア機能の追加）

1. 在庫引当テーブルの新設 (T06相当)

目的: 最小在庫数制約の管理、ボトルネック部品の特定

```
class StockAllocation(models.Model):
    """在庫引当マスタ"""
    product = models.ForeignKey(Product, on_delete=models.CASCADE,
    verbose_name='製品')
    location = models.CharField(max_length=50, verbose_name='保管場所') # 論理的な場所
    current_stock = models.DecimalField(max_digits=14, decimal_places=3,
    verbose_name='現在庫数')
    reserved_qty = models.DecimalField(max_digits=14, decimal_places=3, default=0,
    verbose_name='引当済数量')
    min_stock_qty = models.DecimalField(max_digits=14, decimal_places=3,
    verbose_name='最小在庫数')
    is_bottleneck = models.BooleanField(default=False, verbose_name='ボトルネック品')
    created_at = models.DateTimeField(auto_now_add=True, verbose_name='作成日時')
    updated_at = models.DateTimeField(auto_now=True, verbose_name='更新日時')

    class Meta:
        db_table = 't_stock_allocation'
        verbose_name = '在庫引当'
        verbose_name_plural = '在庫引当'
        unique_together = [['product', 'location']]
```

機能要件:

- 製品ごとの現在庫数、引当済数量、最小在庫数を管理
- 在庫不足時のボトルネック自動判定
- 引当予約機能（製造指示との連携）

2. 製造指示テーブルの新設 (T08相当)

目的: 計画から実行への正式な作業指示書の発行

```
class ProductionOrder(models.Model):
    """製造指示"""
    STATUS_CHOICES = [
        ('PLANNED', '計画済'),
        ('RELEASED', '指示済'),
        ('IN_PROGRESS', '進行中'),
        ('COMPLETED', '完了'),
        ('CANCELED', '中止'),
    ]

    id = models.BigAutoField(primary_key=True)
    order_no = models.CharField(max_length=50, unique=True, verbose_name='製造指示番号')
    product = models.ForeignKey(Product, on_delete=models.CASCADE, verbose_name='製品')
    routing = models.ForeignKey(Routing, on_delete=models.SET_NULL, null=True, blank=True, verbose_name='ルーティング')
    line = models.ForeignKey(Line, on_delete=models.SET_NULL, null=True, blank=True, verbose_name='ライン')
    order_qty = models.DecimalField(max_digits=14, decimal_places=3, verbose_name='指示数量')
    scheduled_start_date = models.DateField(verbose_name='予定開始日')
    scheduled_end_date = models.DateField(verbose_name='予定完了日')
    actual_start_date = models.DateField(null=True, blank=True, verbose_name='実績開始日')
    actual_end_date = models.DateField(null=True, blank=True, verbose_name='実績完了日')
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='PLANNED', verbose_name='ステータス')
    allocation = models.ForeignKey(
        'StockAllocation',
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        verbose_name='在庫引当'
    )
    priority = models.IntegerField(default=0, verbose_name='優先度')
    remark = models.TextField(null=True, blank=True, verbose_name='備考')
    created_at = models.DateTimeField(auto_now_add=True, verbose_name='作成日時')
    updated_at = models.DateTimeField(auto_now=True, verbose_name='更新日時')

    class Meta:
        db_table = 't_production_order'
        verbose_name = '製造指示'
        verbose_name_plural = '製造指示'
        indexes = [
            models.Index(fields=['status']),
            models.Index(fields=['scheduled_start_date']),
        ]
```

機能要件:

- 1セット単位の製造指示ロット番号管理
- ステータス管理（計画済→指示済→進行中→完了）
- 在庫引当との連携
- 優先度管理

3. 工程別製造実績テーブルの新設 (T09相当)

目的: 工程単位での実績収集、「分」ベースの工数管理

```
class ProcessActual(models.Model):
    """工程別製造実績"""
    id = models.BigAutoField(primary_key=True)
    production_order = models.ForeignKey(
        ProductionOrder,
        on_delete=models.CASCADE,
        related_name='actuals',
        verbose_name='製造指示'
    )
    routing_step = models.ForeignKey(
        RoutingStep,
        on_delete=models.CASCADE,
        verbose_name='ルーティング工程'
    )
    process = models.ForeignKey(Process, on_delete=models.CASCADE,
        verbose_name='工程')
    line = models.ForeignKey(Line, on_delete=models.CASCADE, verbose_name='ライン')
    completed_qty = models.DecimalField(max_digits=14, decimal_places=3,
        verbose_name='完了数量')
    actual_duration_min = models.IntegerField(verbose_name='実績工数(分)') # 実績
    工数(分)
    completed_at = models.DateTimeField(verbose_name='完了日時')
    operator = models.CharField(max_length=50, null=True, blank=True,
        verbose_name='作業者')
    remark = models.TextField(null=True, blank=True, verbose_name='備考')
    created_at = models.DateTimeField(auto_now_add=True, verbose_name='作成日時')
    updated_at = models.DateTimeField(auto_now=True, verbose_name='更新日時')

    class Meta:
        db_table = 't_process_actual'
        verbose_name = '工程別製造実績'
        verbose_name_plural = '工程別製造実績'
        indexes = [
            models.Index(fields=['production_order', 'routing_step']),
            models.Index(fields=['completed_at']),
        ]
```

機能要件:

- 工程単位のセット数記録
- 「分」ベースの実績工数収集
- ライン（日）の進捗への反映
- 作業者トレーサビリティ

🕒 優先度：中（設計改善）

4. 工程マスタに管理区分を追加

目的: ライン（日単位管理）と工程（分単位管理）の階層を明確化

```
class Process(models.Model):
    """工程マスタ"""
    # ... 既存フィールド ...

    UNIT_CHOICES = [
        ('DAY', '日単位管理'),
        ('MINUTE', '分単位管理'),
    ]
    management_unit = models.CharField(
        max_length=10,
        choices=UNIT_CHOICES,
        default='MINUTE',
        verbose_name='管理単位'
    )

    # ... 既存フィールド ...
```

実装方法:

1. マイグレーションファイルで `management_unit` フィールドを追加
2. 既存の Process データに対して、デフォルト値 'MINUTE' を設定
3. 必要に応じて、ライン配下の工程は 'DAY' に更新

5. ルーティング設計の見直し（T05相当への接近）

目的: 多品種加工対応の工数管理（同じ工程で異なる製品を加工する場合の工数を柔軟に管理）

```
class ProcessCycleTime(models.Model):
    """部品×工程の標準サイクル時間（多品種対応）"""
    id = models.BigAutoField(primary_key=True)
    product = models.ForeignKey(Product, on_delete=models.CASCADE,
        verbose_name='製品')
    process = models.ForeignKey(Process, on_delete=models.CASCADE,
        verbose_name='工程')
    line = models.ForeignKey(Line, on_delete=models.SET_NULL, null=True,
        blank=True, verbose_name='ライン')
```

```

cycle_time_min = models.DecimalField(
    max_digits=10,
    decimal_places=2,
    verbose_name='標準サイクル時間(分)'
) # 標準サイクル時間(分)
setup_time_min = models.IntegerField(default=0, verbose_name='段取り時間(分)')
# 段取り時間
lot_size = models.IntegerField(default=1, verbose_name='標準ロットサイズ')
is_active = models.BooleanField(default=True, verbose_name='有効')
valid_from = models.DateField(null=True, blank=True, verbose_name='有効開始日')
valid_to = models.DateField(null=True, blank=True, verbose_name='有効終了日')
created_at = models.DateTimeField(auto_now_add=True, verbose_name='作成日時')
updated_at = models.DateTimeField(auto_now=True, verbose_name='更新日時')

class Meta:
    db_table = 'm_process_cycle_time'
    verbose_name = '工程別サイクル時間'
    verbose_name_plural = '工程別サイクル時間'
    unique_together = [['product', 'process', 'line', 'valid_from']]
    indexes = [
        models.Index(fields=['product', 'process']),
    ]

```

移行方法:

1. 既存の `RoutingStep.duration_min` から `ProcessCycleTime` へデータ移行
2. `RoutingStep` は工程の順序管理に特化
3. CRP計算時は `ProcessCycleTime` を参照

CRP（能力所要量計画）計算ロジック:

```

# 所要工数の計算
所要工数（分） = 製造ロット数 × ProcessCycleTime.cycle_time_min

# ライン負荷の計算
# 特定のラインに属するすべての工程の所要工数を集計し、日単位の総負荷を算出

# キャパシティチェック
# 総負荷（分）を、その日の CalendarDay.work_minutes と比較

```

6. 連産品対応の明確化

目的: 連産品（1つの工程で複数の製品が同時に生産される）の管理

```

# Product モデルに追加
class Product(models.Model):
    # ... 既存フィールド ...

```

```

    is_virtual_set = models.BooleanField(
        default=False,
        verbose_name='仮想セット品番'
    )

    # ... 既存フィールド ...

# BOM に連産品フラグ
class BOM(models.Model):
    # ... 既存フィールド ...

    is_coproduct = models.BooleanField(
        default=False,
        verbose_name='連産品BOM'
    )

    # ... 既存フィールド ...

```

運用方法:

1. 連産品（例: A, B, C, D）を生産する場合、仮想セット品番（例: SET-ABCD）を作成
2. `Product.is_virtual_set = True` に設定
3. BOMで SET-ABCD → A, B, C, D の関係を定義
4. `BOM.is_coproduct = True` に設定
5. `ProcessCycleTime` には SET-ABCD のサイクル時間（15分）を設定

● 優先度：低（機能拡張）

7. CRP計算ロジックの実装

実装場所: ビジネスロジックレイヤー（`services/crp_service.py` など）

機能:

- 所要工数の自動計算
- ライン負荷の集計
- キャパシティ超過アラート
- LineBacklog との連携

実装例:

```

# services/crp_service.py
class CRPService:
    def calculate_line_load(self, line_id, target_date):
        """ライン別負荷計算"""
        # 1. 対象ラインの製造指示を取得
        # 2. ProcessCycleTime から所要工数を計算
        # 3. CalendarDay から稼働時間を取得
        # 4. 負荷率を算出
        pass

```

```
def check_capacity(self, line_id, start_date, end_date):
    """キャパシティチェック"""
    # 期間内の各日について負荷率をチェック
    # 100%超過の日を抽出
    pass
```

8. 中間品L/T動的計算

目的: BOMをボトムアップ展開し、中間品のリードタイムを自動計算

計算ロジック:

中間品L/T = MAX(子部品のL/T) + Σ(全工程のサイクル時間 ÷ ライン稼働時間) + 固定待ち時間

実装例:

```
# services/bom_service.py
class BOMService:
    def calculate_intermediate_lt(self, product_id):
        """中間品L/T動的計算"""
        # 1. BOMを再帰的に展開
        # 2. 購入品のL/Tを取得
        # 3. ProcessCycleTime から工程時間を取得
        # 4. ボトムアップで集計
        # 5. Product.standard_lt_days を更新
        pass
```



段階的移行計画

フェーズ1: 基盤整備 (1-2ヶ月)

実装内容:

1. StockAllocation モデル新設
2. ProductionOrder モデル新設
3. ProcessActual モデル新設
4. Process に management_unit フィールド追加
5. マイグレーション実行
6. 基本的なCRUD APIの実装

影響範囲:

- ☒ 新規テーブルなので既存機能への影響は最小
- ⚠ 在庫管理画面の新規開発が必要

成果物:

- ☐ マイグレーションファイル
- ☐ モデル定義 (models.py)
- ☐ 管理画面 (admin.py)
- ☐ API エンドポイント (views.py, serializers.py)
- ☐ フロントエンド画面 (Vue.js)

テスト計画:

- 単体テスト: モデルのバリデーション
- 結合テスト: API の CRUD 動作確認
- E2E テスト: 画面からのデータ登録・更新

フェーズ2: ルーティング改善 (2-3ヶ月)**実装内容:**

1. **ProcessCycleTime** モデル新設
2. 既存 **RoutingStep** データから **ProcessCycleTime** への移行スクリプト作成
3. 多品種加工対応の工数管理UI実装
4. CRP計算ロジックの実装 (services/crp_service.py)
5. LineBacklog との連携強化

影響範囲:

- ⚠ 中～大 (ルーティング関連の既存機能を段階的に移行)
- ⚠ 既存のルーティング参照ロジックの修正が必要

移行手順:

1. **ProcessCycleTime** テーブル作成 (マイグレーション)
2. データ移行スクリプト実行

```
# migration_scripts/migrate_routing_to_cycle_time.py
for step in RoutingStep.objects.filter(duration_min__isnull=False):
    ProcessCycleTime.objects.create(
        product=step.routing.product,
        process=step.process,
        line=step.line,
        cycle_time_min=step.duration_min,
        # ...
    )
```

3. 新しいCRP計算ロジックのテスト
4. 段階的に新ロジックへ切り替え

成果物:

- ☐ ProcessCycleTime モデル
 - ☐ データ移行スクリプト
 - ☐ CRP計算サービス
 - ☐ 工数管理UI
 - ☐ 負荷グラフ表示機能
-

フェーズ3: 高度機能実装 (3-4ヶ月)

実装内容:

1. 連産品対応の完全実装
 - `Product.is_virtual_set` フィールド追加
 - `BOM.is_coproduct` フィールド追加
 - 仮想セット品番の運用ルール確立
2. 中間品L/T動的計算エンジン
 - BOM再帰展開ロジック
 - L/T自動更新バッチ処理
3. ボトルネック分析機能
 - `StockAllocation.is_bottleneck` の自動判定ロジック
 - アラート機能
4. 製造指示～実績のフルワークフロー
 - 製造指示発行画面
 - 実績入力画面
 - 進捗モニタリングダッシュボード

影響範囲:

- ● 大 (システム全体の最適化)
- ⚠ 既存のBOM展開ロジックの大幅な変更

成果物:

- ☐ 連産品管理機能
 - ☐ BOM展開エンジン
 - ☐ L/T自動計算バッチ
 - ☐ ボトルネック分析レポート
 - ☐ 製造指示ワークフロー
 - ☐ 進捗ダッシュボード
-

☒ 実装チェックリスト

フェーズ1

モデル実装

- ☐ `StockAllocation` モデル作成
- ☐ `ProductionOrder` モデル作成
- ☐ `ProcessActual` モデル作成

- ☐ `Process.management_unit` フィールド追加

マイグレーション

- ☐ マイグレーションファイル生成
- ☐ マイグレーション実行（開発環境）
- ☐ マイグレーション実行（本番環境）

API実装

- ☐ StockAllocation の CRUD API
- ☐ ProductionOrder の CRUD API
- ☐ ProcessActual の CRUD API
- ☐ フィルタ・検索機能

UI実装

- ☐ 在庫引当一覧画面
- ☐ 在庫引当登録・編集画面
- ☐ 製造指示一覧画面
- ☐ 製造指示登録・編集画面
- ☐ 工程実績入力画面

テスト

- ☐ 単体テスト
- ☐ 結合テスト
- ☐ E2Eテスト
- ☐ 性能テスト

期待される効果

定量的効果

- **在庫管理精度向上:** 最小在庫数制約により欠品リスク 30% 削減
- **計画精度向上:** CRP機能により負荷超過の事前検知 → 納期遅延 20% 削減
- **工数管理精度向上:** 工程別実績収集により標準時間の精度向上 15%
- **トレーサビリティ向上:** 製造指示～実績の紐付けにより不良原因追跡時間 50% 短縮

定性的効果

- ボトルネック部品の可視化による計画の最適化
- 多品種生産における柔軟な工数管理
- 連産品の正確な原価計算
- 中間品L/Tの動的更新による在庫削減

リスクと対策

リスク1: データ移行の失敗

対策:

- 本番データのバックアップ
- ステージング環境での事前検証
- ロールバック手順の準備

リスク2: 既存機能への影響

対策:

- 段階的なリリース
- 機能フラグによる新旧ロジックの切り替え
- 十分な回帰テスト

リスク3: ユーザー習熟

対策:

- 操作マニュアルの整備
- トレーニングセッションの実施
- 問い合わせ窓口の設置

次のアクション

1. このドキュメントのレビュー

- 関係者によるレビュー
- 要件の追加・修正

2. フェーズ1の詳細設計

- テーブル定義の詳細化
- API仕様書の作成
- UI設計（ワイヤーフレーム）

3. 開発環境の準備

- 開発ブランチの作成
- テストデータの準備

4. 実装開始

- モデル作成
- マイグレーション実行
- API実装